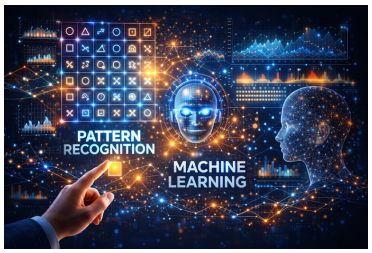
Introduction to Machine Learning
CSL2050 - Sem 02 – 2026
Lecture 25-26-28: Intro to Convolutional NN

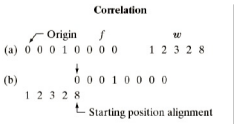



Avinash Sharma
3DVisLab, CSE, IIT Jodhpur

IC: ChatGPT

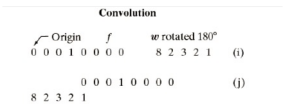
1

## Correlation



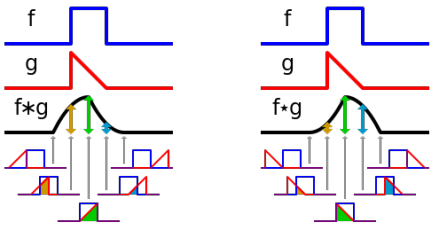

2

## Convolution




3

## Convolution vs Correlation



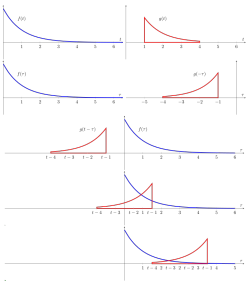

Convolution Correlation

Both converge if signals involved are symmetric !

4

## Convolution Operator

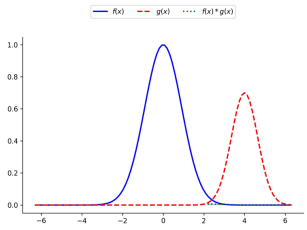
$$(f * g)(t) = \int_{-\infty}^{\infty} f(\tau)g(t-\tau)d\tau$$



5

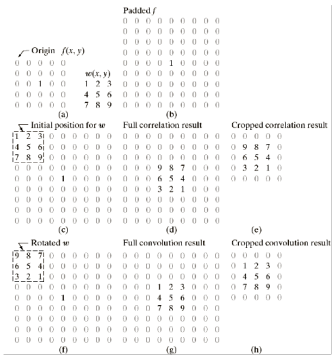
## Convolution Operator

$$(f * g)(t) = \int_{-\infty}^{\infty} f(\tau)g(t-\tau)d\tau$$




6

## Convolution vs Correlation (2D)

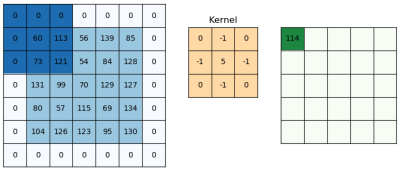



7

## Convolution (2D)

$$w(x,y) \star f(x,y) = \sum_{s=-a}^{a} \sum_{t=-b}^{b} w(s,t) f(x-s, y-t)$$

- Evaluated for all values of displacement variables $x$ and $y$
- Filter size $m \times n$ (notational convenience → $m, n$ are assumed odd)
- $a = (m-1)/2$ and $b = (n-1)/2$

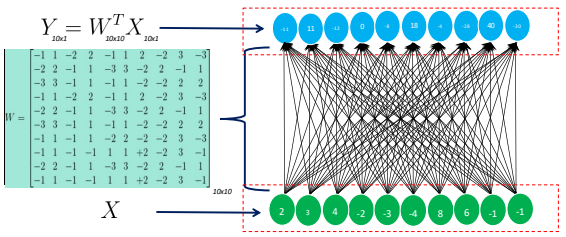



8

Example: 1D Convolution layer

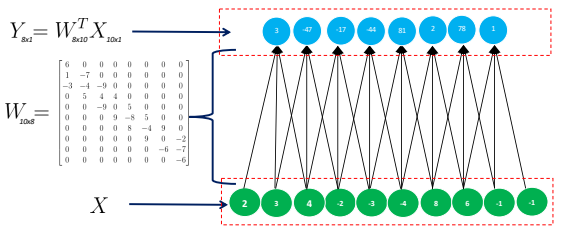
9

Fully Connected NN, Dense connections (10 X 10 = 100)



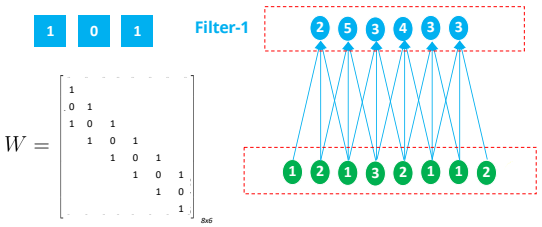

10

What if connections are only local? (8X3 = 24)



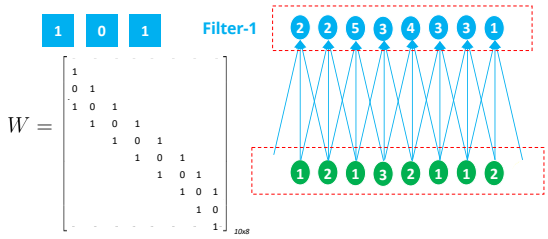

11

What if weights are same/shared? (1 X 3 = 3 !! )




13

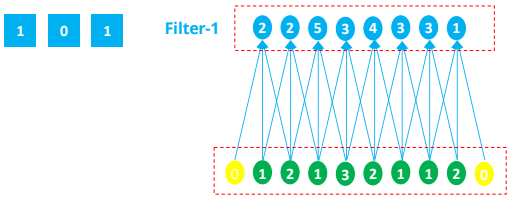
What if weights are same/shared? (1 X 3 = 3 !! )




14

14

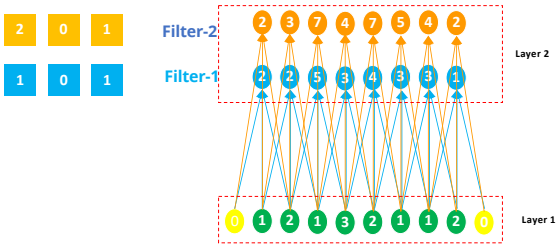
What if weights are same/shared? (1 X 3 = 3 !! )




15

15

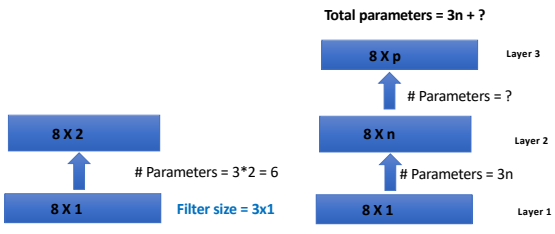
Two such filters/weights (2 X 3 = 6 !! )




16

16

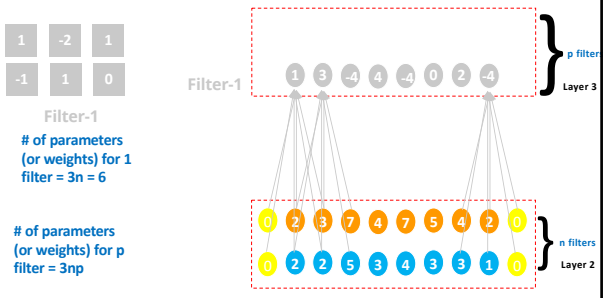
Pictorially




17

17

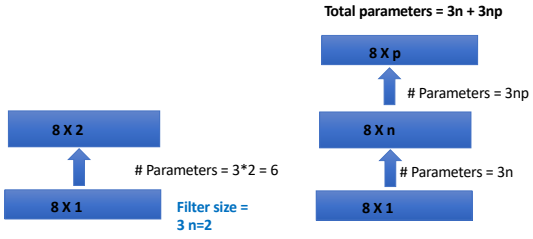
## Spatially varying filters/weights (2 X 3 = 6 !! )




18

18

## Pictorially




19

19

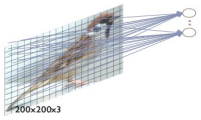
## Convolution layer in 2D (popular)

20

20

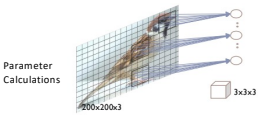
## Convolution layer

- Fully connected layer




- Image of size 200 X 200 and 3 colours (RGB)
- #Hidden Units: 120,000 (= 200X200X3)
- #Params: 14.4 billion (= 120K X 120K)
- Need huge training data to prevent over-fitting!

- Locally connected layer



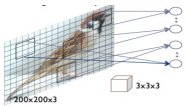

- #Hidden Units: 120,000
- #Params: 3.2 Million (= 120K X 27)
- Useful when the image is highly registered

21

21

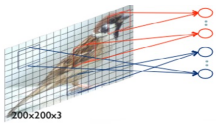
## Convolution layer

- Convolutional layer with **single** feature map.

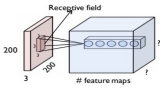



- #Hidden Units: 120,000
- #Params: 27
- **Sharing parameters**
- Exploiting the ***stationarity*** property and preserves locality of pixel dependencies

- Convolutional layer with **multiple** feature maps




- #Params: 27 x #Feature Maps

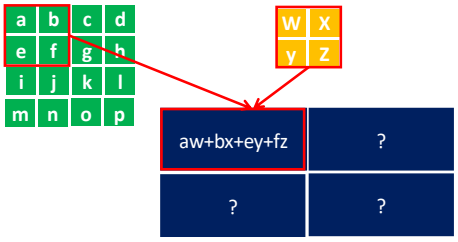



In the context of CNNs, ***stationarity*** refers to how separate neighbourhoods of pixels tend to display similar patterns. This is why the *same* set of filters can be used to capture local correlations at different locations in the input image, and at different locations across multiple input images.

22

22

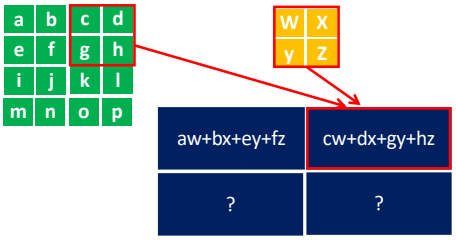
## Convolution in 2-D




23

23

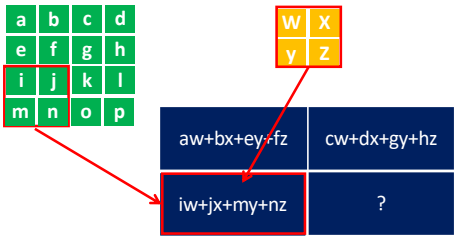
## Convolution in 2-D




24

24

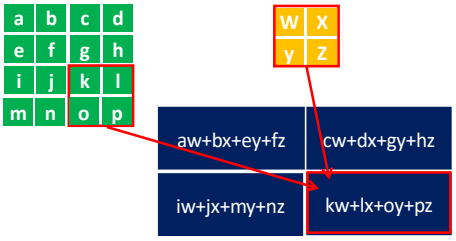
## Convolution in 2-D




25

25

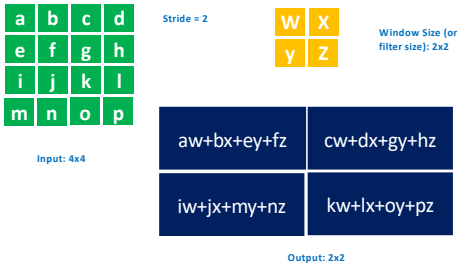
## Convolution in 2-D




26

26

## Convolution in 2-D

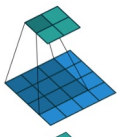



27

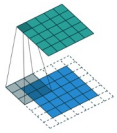
27

## CNNs

- Window size (FxF)
- Stride
- Padding



Input: 4x4
Window size: 3x3
Stride: 1
Padding: 0
Output: 2x2



Input: 5x5
Window size: 3x3
Stride: 1
Padding: 1
Output: 5x5

28

28

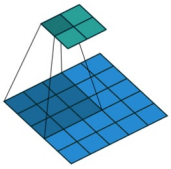
## CNNs

- Larger stride reduces dimension

$W_2 = [\,(W_1 - F + 2P)/S\,] + 1$

$H_2 = [\,(H_1 - F + 2P)/S\,] + 1$

**Output: $W_2$ x $H_2$**
$W_2 = [\,(5-3+0)/2\,] + 1 = 2$
$H_2 = [\,(5-3+0)/2\,] + 1 = 2$



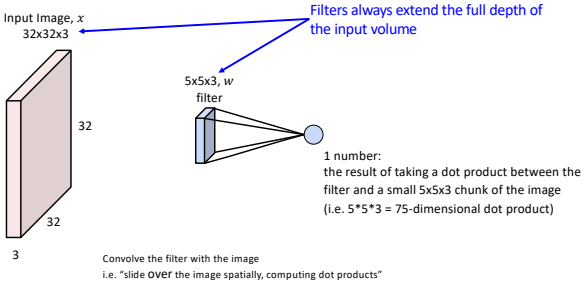
Input ($W_1$ x $H_1$): 5x5
Window size ($F$ x $F$): 3x3
Stride ($S$): 2
Padding ($P$): 0
Output ($W_2$ x $H_2$): 2x2

29

29

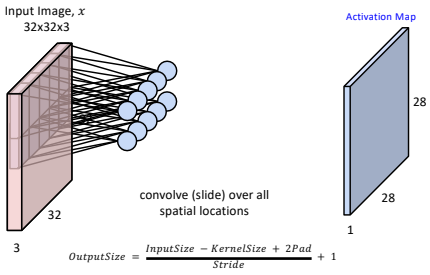
## CNN

How to generalize to Color Images ?



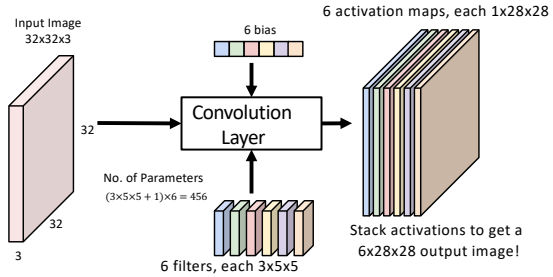

30

## Convolution



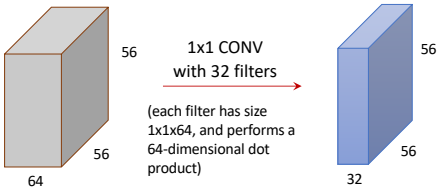

31

## Convolution Layer




32

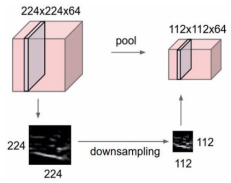
Note: 1x1 convolutions are perfectly fine




33

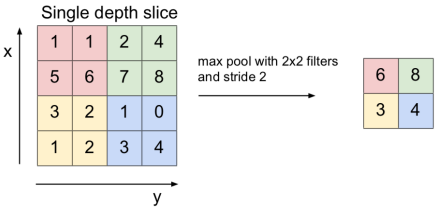
## Pooling Layer

- Makes representations smaller and manageable
- Later filters have larger support
- Operates over each activation map independently




34

## Max Pooling (2D)




Other Pooling options like average, min, sum pooling exists

35

## Fully connected

- Multi layer perceptron
- Role of a classifier
- Generally used in final layers to classify the object represented in terms of discriminative parts and higher semantic entities.
- SoftMax
  - Normalizes the output.
  - K is total number of classes

$$z_n = \frac{e^{x_n}}{\sum_{i=1}^{K} e^{x_i}}$$

36

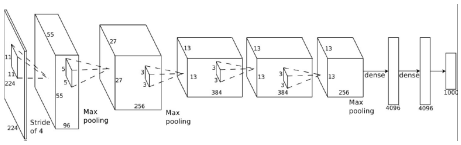
36

## CNN for Images

37

37

## AlexNet




"95% Parameters in FC but 95% Computation is done for learning the 5% Convolution weights !"

About 57 M parameters are in the fully connected layers

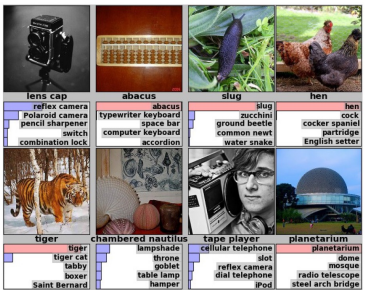
| | | | | | | | | | |
|---|---|---|---|---|---|---|---|---|---|
| *Parameters :* | [(11 x 11 x 3 ) + 1] x 96 = 35 K | [ 5 x 5 x 48 ] x 256 = 307 K | [ 3 x 3 x 256 ] x 384 = 884 K | 663 K | 442 K | 37 M | 16 M | 4 M | *Total* 60 M |
| *Neurons :* | 253,440 | 27 x 27 x 256 = 186,624 | 13 x 13 x 384 = 64,896 | 13 x 13 x 384 = 64,896 | 13 x 13 x 256 = 43,264 | 4096 | 4096 | 1000 | 0.63 M |

Imagenet classification with deep convolutional neural networks. Krizhevsky, Alex, IlyaSutskever, and Geoffrey E. Hinton. *NIPS* 2012.



38

## AlexNet Results

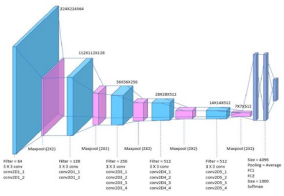





39

## Visual Geometry Group's (VGG) Model

- Used small (3x3) convolutional filters
  - Captures fine-grain details
  - Can learn more complex & abstract features
  - Uses fewer parameters
- With more layers, there are more opportunities for non-linear activations
- Proposed Variants (#layers): VGG-13, VGG-16, VGG-19

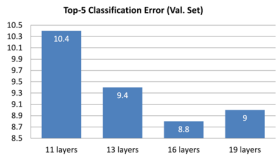


Simonyan, Karen and Andrew Zisserman. "Very Deep Convolutional Networks for Large-Scale Image Recognition." CoRR abs/1409.1556 (2014)

40

## Deeper Nets are always better ?

- More layers lead to more non-linearities
- Smaller receptive fields:
  - less parameters; faster
- Nevertheless, more training data needed for deeper networks

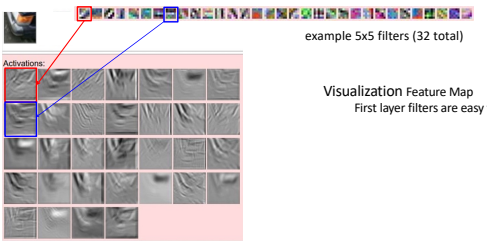



Empirically
Not Always.



41

## Visualizing the CNN features !

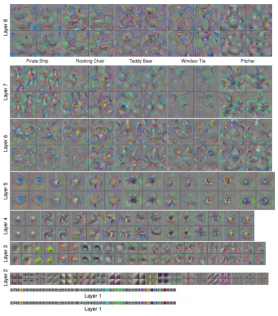



example 5x5 filters (32 total)

Visualization Feature Map
First layer filters are easy to visualize.

42

## Visualizing the CNN features !



- Initial layers capture lower level constructs like edges, corners, (texture)and blobs (color).
- Higher layers captures higher level semantic constructs in both texture and color space.

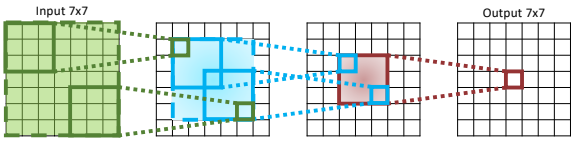
**Image Source Lipson et. al. arXiv:1506.06579**

4 random gradient descent runs for each channel

44

## Receptive Field

- Each successive convolution adds $K - 1$ to the receptive field size With $L$ layers the receptive field size is $1 + L \times (K - 1)$

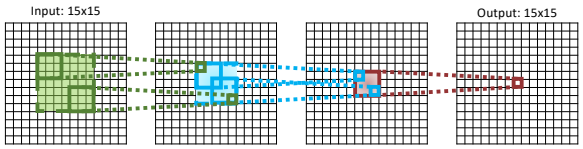



Kernel: 3x3
Pad: 1
Stride: 1

45

## Receptive Field of Large Input

Problem: For large images we need many layers for each output to "see" the whole image

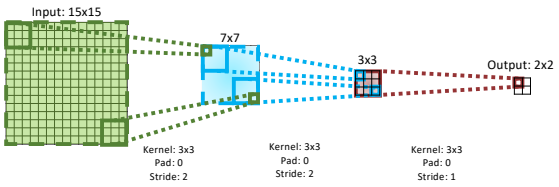



Kernel: 3x3
Pad: 1
Stride: 1

Solution: Strided Convolutions, Subsampling

46

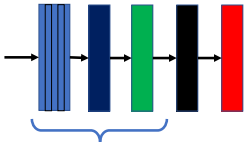
## Receptive Field with Strided Convolution




47

## Summary: Layers of a CNN

- Based on the connection pattern and operations, we can think of a layer in a Neural Network as:
  - Convolutional
    - A Layer can have multiple Channels
  - Non-Linear (often not drawn)
  - Max-Pooling
  - Fully Connected
  - Soft Max



This is often repeated multiple times

48

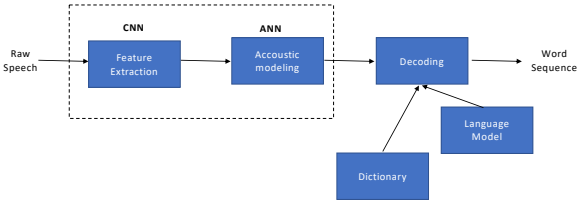
48

## Convolution for speech

49

49

## Speech Recognition model

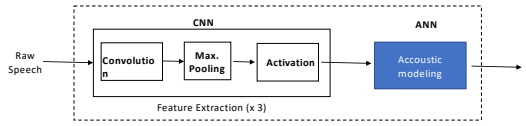



Dimitri Palaz, Mathew Magimai-Doss and Ronan Collobert, "Convolutional Neural Networks-based Continuous Speech Recognition using Raw Speech Signal", ICASSP 2015, pp.4295-4299.

50

50

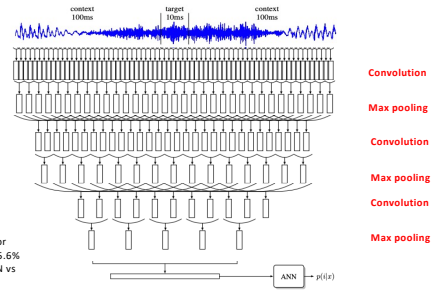
## Speech Recognition model




51

51

## Detailed View



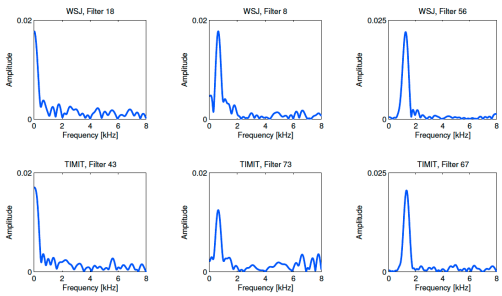

WSJ Word Error Rate: 64.% vs 5.6% for MFCC+ANN vs CNN

https://pdfs.semanticscholar.org/d224/6d41fe4ca1cb1f13eaa7bec0b86b95fdd8ac.pdf

52

52

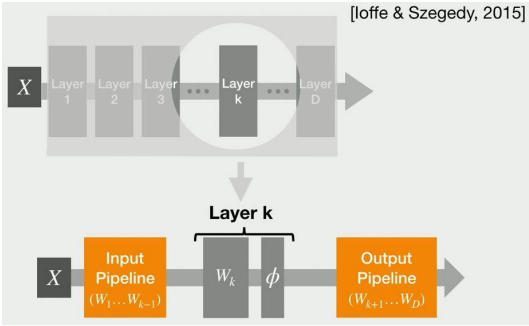
## Comparing Learnt Filters: WSJ vs. TIMIT




53

53

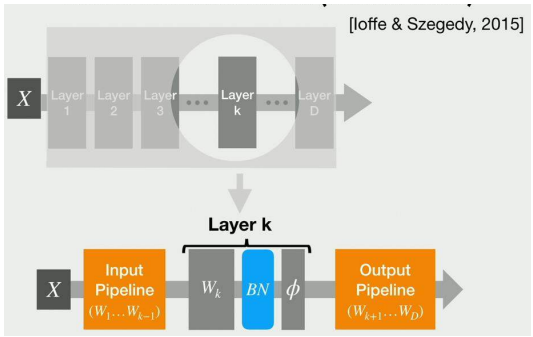
## Batch Normalization




54

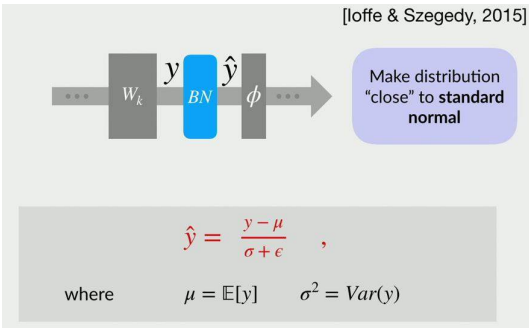
54

## Batch Normalization



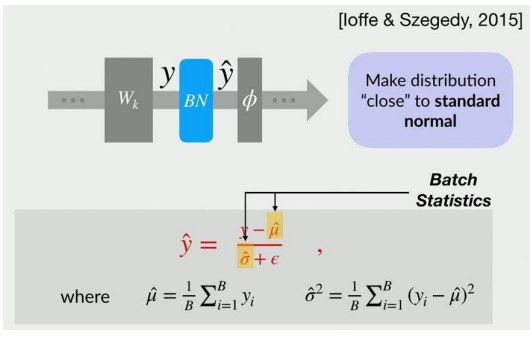

55

## Batch Normalization



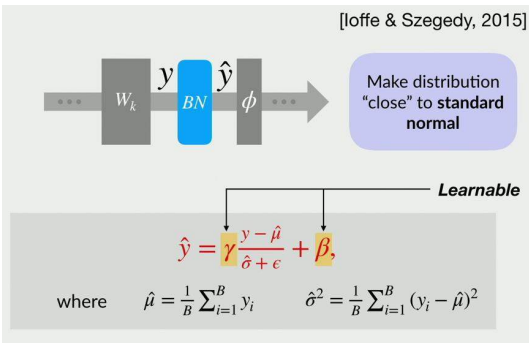

56

## Batch Normalization




57

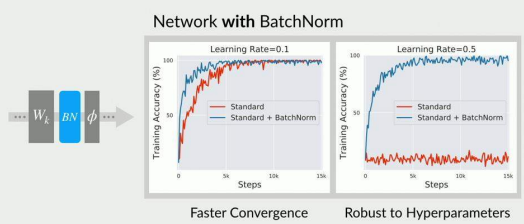
## Batch Normalization




58

## Batch Normalization

### How it helps ?



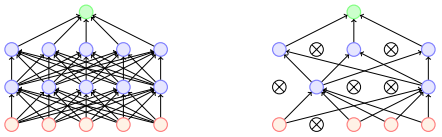

Know more at: https://www.microsoft.com/en-us/research/wp-content/uploads/2019/05/How-does-Batch-Normalization-Help-Optimization-slides.pdf

59

59

## Dropout as Regularizer

- Dropout refers to dropping out units.
- Temporarily remove a node and all its incoming/outgoing connections resulting in a thinned network.
- Each node is retained with a fixed probability (typically p = 0.5 for hidden nodes and p = 0.8 for visible nodes )

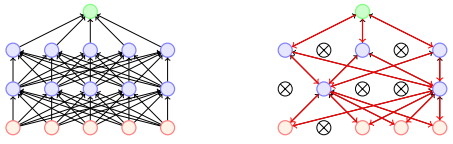


60

60

## Dropout as Regularizer

- We initialize all the parameters (weights) of the network and start training.
- For the first training instance (or mini-batch), we apply dropout resulting in the thinned network.
- We compute the loss and backpropagate Which parameters will we update? Only those which are active

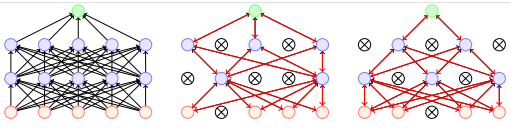


61

61

## Dropout as Regularizer

- For the second training instance (or mini-batch), we again apply dropout resulting in a different thinned network.
- We again compute the loss and backpropagate to the active weights.
- If the weight was active for both the training instances then it would have received two updates by now.
- If the weight was active for only one of the training instances then it would have received only one updates by now.

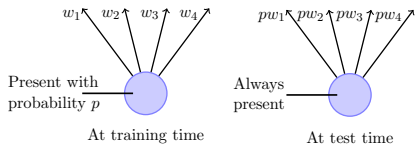


62

62

## Dropout as Regularizer

- What happens at test time?
- We use the full Neural Network and scale the output of each node by the fraction of times it was on during training.




63

63

## Dropout as Regularizer

- Dropout essentially applies a masking noise to the hidden units.
- Prevents hidden units from co- adapting.
- Essentially a hidden unit cannot rely too much on other units as they may get dropped out any time.
- Each hidden unit has to learn to be more robust to these random dropouts.
- This reduces chance of overfitting as well as make the network more robust to deal with noisy/missing data.

Know more at : https://www.cse.iitm.ac.in/~miteshk/CS7015/Slides/Teaching/src/Lecture8/modules/Module11/Lecture8_11.pdf

64

64

## Thank you

65

65